Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport
la
Lucrarea de laborator Nr. 7

Disciplina: Tehnici de programare

Tema: ” Trusa de unelte HTTP ”

A efectuat : st. gr.SI – 261 Grecu Gabriela

A verificat : asistent universitar Marius Dumitrascu

Chișinău – 2026

SARCINA LUCRĂRII

Obiectiv: Construirea pas cu pas a unui set de funcții utilitare utilizând biblioteca `requests` din Python pentru a comunica cu un server web, gestiona parametri, anteturi, trimitere de date (JSON/formular), coduri de stare, redirecționări, sesiuni/cookie-uri și tratarea robustă a erorilor.

Partea 1: Parametri în URL

- **Scop:** Trimiterea de parametri către un server prin intermediul adresei web, fără a construi manual șirul de caractere (evitând astfel problemele legate de spații sau caractere speciale precum `&`).
- **Pași de lucru:**
  - Se utilizează funcția de cerere de tip `GET` către adresa de bază, completată cu ruta specifică pentru obținerea datelor.
  - Se transmite un dicționar de date prin intermediul argumentului dedicat pentru parametri.
  - Pentru prima cerință, se extrage din răspunsul în format JSON secțiunea care conține argumentele primite de server.
  - Pentru a doua cerință, se solicită direct adresa completă (URL-ul final) generată automat de bibliotecă, care include parametrii codificați corect în șirul de interogare (*query string*).

Partea 2: Anteturi (Headers)

- **Scop:** Controlarea modului în care clientul se identifică în fața serverului prin intermediul antetelor HTTP, în special `User-Agent`.
- **Pași de lucru:**
  - Se trimite o cerere de tip `GET` către pagina dedicată de afișare a anteturilor.
  - Pentru prima funcție, se adaugă un dicționar care setează antetul `User-Agent` la o valoare personalizată (de exemplu, un nume de robot sau aplicație), iar serverul va returna exact valoarea pe care a recepționat-o.
  - Pentru a doua funcție, cererea se trimite fără a specifica niciun antet suplimentar. În acest caz, serverul va înregistra valoarea implicită cu care biblioteca de cereri se identifică în mod standard.

Partea 3: Trimiterea datelor (POST)

- **Scop:** Trimiterea de date către un server prin intermediul corpului (*body*) unei cereri de tip `POST`, utilizând fie formatul structurat JSON, fie formatul clasic de formular web.
- **Pași de lucru:**
  - Se utilizează o cerere de tip `POST` către ruta destinată primirii datelor.
  - Atunci când datele se transmit ca obiect structurat, se folosește argumentul dedicat pentru JSON, care va serializa datele și va seta automat antetul de tip `application/json`.
  - Atunci când datele se transmit sub formă de formular, se folosește argumentul dedicat pentru date de formular, care va trimite conținutul codificat corespunzător.
  - Serverul va returna în corpul răspunsului secțiunea corespunzătoare formatului în care a primit datele, care va fi extrasă și returnată de funcție.

Partea 4: Coduri de stare HTTP

- **Scop:** Interpretarea răspunsurilor primite de la server în funcție de codurile numerice de stare și tratarea sigură a erorilor pentru a preveni oprirea bruscă a programului.
- **Pași de lucru:**
  - Se efectuează o cerere `GET` către o rută care simulează returnarea unui anumit cod de stare numeric.
  - Pentru prima funcție, codul de stare primit este clasificat pe intervale: intervalul de succes (sutele doi), erori generate de client (sutele patru) sau erori generate de server (sutele cinci), returnând un mesaj text corespunzător.
  - Pentru a doua funcție, se folosește o metodă nativă care declanșează o excepție în cazul în care serverul raportează o eroare de tip client sau server. Această excepție este captură într-un bloc dedicat de tratare a erorilor astfel încât funcția să returneze `True` dacă totul este în regulă sau `False` dacă a apărut o eroare, fără a opri execuția aplicației.

Partea 5: Redirecționări

- **Scop:** Gestionarea comportamentului serverului atunci când acesta trimite clientul către o altă adresă web.
- **Pași de lucru:**
  - Se accesează o rută care execută în mod repetat redirecționări înainte de a ajunge la destinația finală.
  - În mod normal, biblioteca urmează automat aceste redirecționări, păstrând întreg istoricu pașilor parcurși. Numărul de elemente din istoricul cererii indică exact câte redirecționări au avut loc.
  - Pentru a afla adresa finală la care se ajunge, se citește proprietatea corespunzătoare adresei URL după finalizarea tuturor salturilor.

- Pentru a intercepta prima redirecționare fără a lăsa programul să o urmeze automat, cererea se configurează cu un parametru care dezactivează urmărirea automată a redirecționărilor. Astfel, răspunsul se oprește la primul pas, permițând citirea antetului de locație care indică unde dorea serverul să trimită clientul.

Partea 6: Sesiuni și Cookie-uri

- **Scop:** Înțelegerea modului în care se păstrează starea și cookie-urile între cereri succesive prin utilizarea unui obiect de tip sesiune.
- **Pași de lucru:**
  - Se utilizează o cerere pentru a seta un cookie pe server.
  - Într-o variantă care folosește o sesiune unică, obiectul de sesiune reține automat cookie-ul primit de la server la prima cerere și îl trimite înapoi la a doua cerere, permițând serverului să-l recunoască.
  - În varianta care folosește cereri simple și independente, fiecare apel pornește de la zero și nu reține nimic din interacțiunile anterioare. Prin urmare, la a doua cerere, serverul nu va mai vedea cookie-ul setat anterior, dovedind utilitatea menținerii unei sesiuni.

Partea 7: Raportul final și Bucla de procesare

- **Scop:** Combinarea tuturor conceptelor într-o funcție complexă de raportare și automatizarea parcurgerii unei liste de rute.
- **Pași de lucru:**
  - **Funcția de raportare:** Trimite o cerere către o adresă web având impus un timp limită (*timeout*) strict pentru a preveni blocarea în cazul unor servere lente. Se extrag indicatori precum codul de stare, adresa finală, numărul de redirecționări, durata măsurată a cererii și tipul conținutului. Orice eroare de rețea sau de timp depășit este capturată în mod controlat, returnând un dicționar cu starea nulă și mesajul erorii, fără a opri aplicația.
  - **Bucla principală:** Se parcurge o listă predefinită de rute, aplicând funcția de raportare fiecăreia.
  - **Filtrare și sortare:** Se selectează doar paginile care au întors un succes complet (status 200), acestea fiind ordonate descrescător în funcție de timpul de răspuns (cea mai lentă primă).
  - La final, se centralizează și se afișează statistici privind numărul total de pagini care au reușit și cele care au eșuat.

CONCLUZII

În urma realizării acestei lucrări de laborator, am parcurs etapele fundamentale pentru construirea unei interacțiuni robuste și controlate cu un server web folosind biblioteca Python `requests`. Exercițiul a demonstrat importanța utilizării parametrilor structurați și a anteturilor personalizate în transmiterea corectă a datelor, precum și relevanța alegerii metodelor potrivite pentru trimiterea informațiilor în corpul cererilor, fie sub formă de obiecte JSON, fie ca formulare web. De asemenea, gestionarea riguroasă a codurilor de stare și a excepțiilor prin blocuri de siguranță s-a dovedit esențială pentru a preveni oprirea bruscă a programului în cazul unor erori de rețea sau al unor răspunsuri de tip client și server. Nu în ultimul rând, utilizarea sesiunilor pentru menținerea stării și a cookie-urilor, alături de automatizarea procesării printr-o buclă de raportare dotată cu limite de timp, a evidențiat avantajele creării unui cod modular, sigur și orientat spre rezilience în fața unor servicii web indisponibile sau lente.

BIBLIOGRAFIE

1.